Working with lists of objects

Today

Welcome to your next lesson: Working with lists of objects.

I'm getting your personalized lesson ready — based on what you've learned, what's coming up, and how you learn best.

Step by step, you’re moving forward.

Imagine you already have several Pet objects created: a dog, a cat, a parrot 🐶🐱🦜

In this lesson, we’ll use a list of those pet objects and a for loop to go through them one by one and use their data and methods.

Quick check-in to start:

When you see a loop like this:

Python
```
for pet in pets:
    print(pet.name)
```
Do you already feel pretty comfortable with basic for loops over lists in Python?

Yes
Kind of
Not really yet
Yes

What do you think? 👇

A list contains three pet objects. During one pass through for pet in pets, what does pet represent?

A.
The entire list

B.
The current pet object

C.
The pet class definition

D.
The output produced by the loop

Nice work, Ron 🎉 You picked the right answer: pet is the current pet object during that one pass through the loop.

Step by step, here’s what happens in:

Python
```
for pet in pets:
    print(pet.name)
```
pets → the whole list (all the pets together)
pet → just one pet from that list on this loop step
pet.name → the name attribute of that one pet

So on each pass, pet changes to the next pet object in the list.

Ready to see this in action with a small pet example and then try a tiny task in the code editor?

Yes, show me the example

Nice, let’s build it step by step 🐾

We’ll imagine a simple Pet class. Don’t worry about the details of the class too much — focus on the loop over the list of objects.

Here’s the idea in plain language first:

We have 3 pet objects: a dog, a cat, a parrot.
We put them in a list called pets.
We use a for loop. On each step:
pet is one of those objects

```
we print some data from that object (like its name)
we call a method on that object (like speak())

Visually:


[pet1, pet2, pet3]  -->  pet (current) -->  pet.name / pet.speak()

Now, here’s what I want you to do in the code editor (not in chat):


Open the editor (</> button).
Type this code:
Python
class Pet:
    def __init__(self, name, species):
        self.name = name
        self.species = species

    def speak(self):
        print(self.name, "says hello!")


pet1 = Pet("Buddy", "dog")
pet2 = Pet("Mittens", "cat")
pet3 = Pet("Polly", "parrot")

pets = [pet1, pet2, pet3]

for pet in pets:
    print("Pet name:", pet.name)
    pet.speak()
First, run it (Run button) and look at the output.
Then Submit code so I can see it and we can talk about what happened.

Go ahead and do that, then I’ll ask you to explain what pet is doing in this loop, in your own words 💡

Python
class Pet:
    def __init__(self, name, species):
        self.name = name
        self.species = species

```

```
    def speak(self):
        print(self.name, "says hello!")


pet1 = Pet("Buddy", "dog")
pet2 = Pet("Mittens", "cat")
pet3 = Pet("Polly", "parrot")

pets = [pet1, pet2, pet3]

for pet in pets:
    print("Pet name:", pet.name)
    pet.speak()
Love it, Ron — your code and output are perfect ✅


You just did two key things with a list of objects:


Accessed data on each object:
print("Pet name:", pet.name)
Called behavior (a method) on each object:
pet.speak()

On each loop step, pet was one of pet1, pet2, or pet3, in
order.

Quick check, in your own words (one short sentence):

During the loop for pet in pets:, what is pet at each step?

pet1 then pet 2 then pet 3. Each individual pet object

Take your time and answer the question below 👇

A list contains three pet objects. During one pass through
for pet in pets, what does pet represent?


A.
The entire list


B.
The current pet object


```

C.
The pet class definition

D.
The output produced by the loop

Nice, that’s exactly right 🎯

In for pet in pets::

pets is the whole list
pet is the current pet object for that one turn of the loop (so first pet1, then pet2, then pet3)

That’s why pet.name and pet.speak() work — they’re using the attributes and methods of that one object at a time.

Next small step: I want to show you a different example (not pets), then have you do a tiny practice.

We’ll use game characters 🎮

I’ll show you some code in the next message. Ready to look at that example?

Yes, show me the example

Let’s switch to game characters now 🎮

Imagine we already have this class:

Python
```
class Character:
    def __init__(self, name, level):
        self.name = name
        self.level = level

    def introduce(self):
        print("I am", self.name, "and my level is", self.level)
```
And we create a few characters and put them in a list:

```
Python
hero = Character("Aria", 5)
mage = Character("Doran", 8)
rogue = Character("Silk", 3)

party = [hero, mage, rogue]

for member in party:
    print(member.name, "is level", member.level)
    member.introduce()
```

Step by step in the loop:

party → the whole list of character objects
member → one Character object at a time (first hero, then mage, then rogue)
We read data: member.name, member.level
We call behavior: member.introduce()
Make sense how this is the same pattern as the pets example, just with different objects?

If yes, I'll give you a small practice task with characters next.

Yes, that makes sense

What do you think member is in that loop? 👇

A list contains three game character objects. During one pass through for member in party, what does member represent?

A.
The entire party list

B.
The current character object

C.
The Character class definition

D.
The printed output so far

What do you think? Pick your answer below 👇

You unlocked the Collection Champion badge! What can a loop do with a list of objects?

A.
Process each object one at a time

B.
Turn every object into a class

C.
Automatically create user input

D.
Make all objects identical

You nailed it, Ron 🎯

Your answer — “Process each object one at a time” — is exactly what a loop does with a list of objects.

Step by step:

The list holds many objects (pets, characters, whatever you model).
The for loop walks through that list.
On each step, the loop variable (pet, member, etc.) is one object.
Inside the loop, you can:
read its data (pet.name, member.level)
call its methods (pet.speak(), member.introduce())

That’s the core idea of this lesson, and you’ve shown you can use it in code and explain it in words 💪

You’ve mastered:

iterating over a list of objects
accessing attributes inside the loop
calling methods on each object in the collection

Whenever you’re ready, you can move on to the next lesson in the Learn tab.

If you want, you can also quickly tell me how you might use this pattern (e.g., tasks, customers, enemies in a game), and I’ll suggest a custom mini-practice for it.